

Reviewer Pack — Minimal Order of Geometric Entanglement and Communication Co...

Entry 65c8aa488cc5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 14:39:31 UTC

Minimal Order of Geometric Entanglement and Communication Complexity Rank

Entry ID: 65c8aa488cc5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 14:39:31 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Geometric Entanglement) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -ary communication complexity function $f(n)$, the minimal geometric entanglement $E(G_f)$ of the associated quantum state G_f is upper bounded by a constant C such that $E(G_f) \leq C \log^2(n)$.

Rationale (proposer's reasoning):

Geometric entanglement measures the non-classical nature of a quantum state, and its connection to communication complexity could reveal a new perspective on proving lower bounds in communication complexity. A strong upper bound on geometric entanglement would imply a relationship between entanglement and the difficulty of classical computation.

Taxonomy category: quantum_information_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c02fb3fa10cfb8cb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every n -ary communication complexity function, the computed minimal geometric entanglement $E(G_f)$ should not exceed $C \log^2(n)$, where C is a constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric entanglement" AND "communication complexity" AND "matrix rank" - "quantum information theory" AND "geometric entanglement" AND "upper bound on entanglement" - "entanglement complexity" AND "communication complexity" AND "n-ary function"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i + 1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
```

```

    return C

def construct_quantum_state(f_n, n):
    G_f = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1):
            G_f[i][j] = f_n[i ^ j]
            G_f[j][i] = G_f[i][j]
    return G_f

def minimal_geometric_entanglement(G_f, n):
    I = [[Fraction(1 if i == j else 0) for j in range(n)] for i in range(n)]
    A = matrix_multiply(G_f, I)
    U, _, Vt = gaussian_elimination(A)
    entanglement = sum(sum(abs(x)) for x in U) * sum(sum(abs(x)) for x in Vt)
    return entanglement

def communication_complexity_rank(f_n, n):
    # Placeholder function; replace with actual implementation
    return len(f_n)

n_values = [5, 10, 15, 20, 30, 40]
metric_value = 0.0
instances_tested = 0
n_max = 0

for n in n_values:
    f_n = [random.random() for _ in range(2**n)]
    G_f = construct_quantum_state(f_n, n)
    entanglement = minimal_geometric_entanglement(G_f, n)
    rank = communication_complexity_rank(f_n, n)

    metric_value += entanglement
    instances_tested += 1
    if n > n_max:
        n_max = n

mean_entanglement = Fraction(metric_value, instances_tested)
conjecture_holds = all(mean_entanglement <= C * math.log(n) ** 2 for n in n_values)

return {
    "metric_name": "Minimal Geometric Entanglement",
    "metric_value": float(mean_entanglement),
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f58223e.py", line 101, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f58223e.py", line 75, in run_trial
    entanglement = minimal_geometric_entanglement(G_f, n)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9f58223e.py", line 59, in minimal_geome
    U, _, Vt = gaussian_elimination(A)
    ^^^^^^^
ValueError: too many values to unpack (expected 3)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the computation of the minimal geometric entanglement $E(G_f)$. As a result, we cannot verify whether the conjecture is supported or falsified. | next: Re-run the test with appropriate error handling to ensure that it can complete its execution and produce results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12995
2	preregistration	ollama_remote	glm4:latest	0	0	9542
3	novelty	ollama_remote	glm4:latest	0	0	8504
4	novelty	ollama_remote	glm4:latest	0	0	9357
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16437
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12388
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13231
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11741
9	judge	ollama_remote	glm4:latest	0	0	15339

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 109534 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/65c8aa488cc5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/65c8aa488cc5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/65c8aa488cc5.tar.gz` (if generated)